



Rexx

Rexx (restructured extended executor) is a high-level programming language developed at IBM by Mike Cowlishaw.^{[6][7]} Both proprietary and open source Rexx interpreters exist for a wide range of computing platforms, and compilers exist for IBM mainframe computers.^[8] Rexx is used for scripting, application macros and application development. As a general purpose scripting language, Rexx is considered a precursor to Tcl and Python.

Rexx is supported in a variety of environments. It is the primary scripting language in some operating systems including OS/2, MVS, VM, AmigaOS and is used for macros in some software including SPF/PC, KEDIT, THE and ZOC. With an engine installed, Rexx can be used for scripting and macros in programs that use a Windows Scripting Host ActiveX scripting engine (such as VBScript or JScript). Rexx is supplied with VM/SP Release 3 on up, TSO/E Version 2 on up, OS/2 (1.3 and later, where it is officially named *Procedures Language/2*), AmigaOS Version 2 on up, PC DOS (7.0 or 2000), ArcaOS,^[9] and Windows NT 4.0 (Resource Kit: Regina). In the late 1980s, Rexx became the common scripting language for IBM Systems Application Architecture, where it was renamed "SAA Procedure Language REXX".

A script is associated with a Rexx interpreter at runtime in various ways based on context. In mainframe computing, a Rexx script or command is sometimes referred to as an *EXEC* since that is the name of the file type used for similar CMS EXEC,^[10] and EXEC 2^[11] scripts and for Rexx scripts on VM/SP R3 through z/VM. The first line of a script specifies the use of a Rexx interpreter in a comment either by identifying the code as Rexx language or by file path via EXTPROC. On MVS, Rexx scripts may^[a] be recognized by the low level qualifier "EXEC" or if the first line fetched from SYSPROC is a comment containing "REXX" then it is treated as Rexx (rather than CLIST), and a script fetched from SYSEXEC must be Rexx. On OS/2, Rexx scripts share the filename extension ".cmd" with other scripting languages, and the first line of the script specifies the interpreter to use. On Linux, Rexx scripts generally begin with a shebang. Rexx macros for Rexx-aware applications use extensions determined by the application.

Rexx	
	
Paradigm	<u>multiparadigm</u> : <u>procedural</u> , <u>structured</u>
Designed by	<u>Mike Cowlishaw</u>
Developer	<u>Mike Cowlishaw</u> , <u>IBM</u>
First appeared	1979
Stable release	<u>ANSI X3.274 / 1996</u>
Typing discipline	<u>Dynamic</u>
Filename extensions	<u>.cmd</u> , <u>.bat</u> , <u>.exec</u> , <u>.rex</u> , <u>.rex</u> , <u>EXEC</u>
Major implementations	
<u>VM/SP R3</u> , ^[1] <u>TSO/E V2</u> , ^[2] <u>SAAREXX</u> , ^{[3][4]} <u>ARexx</u> , <u>BREXX</u> , <u>Regina</u> , ^[5] <u>Personal REXX</u> , <u>REXX/imc</u>	
Dialects	
<u>NetRexx</u> , <u>Object REXX</u> , now <u>ooREXX</u> , <u>KEXX</u>	
Influenced by	
<u>PL/I</u> , <u>ALGOL</u> , <u>EXEC</u> , <u>EXEC 2</u>	
Influenced	
<u>NetRexx</u> , <u>Object REXX</u>	
 <u>Rexx Programming at Wikibooks</u>	

Name

Originally, the language was called *REX*, short for *Reformed Executor*, but an extra "X" was added to avoid confusion with other products. The name was originally all uppercase because that was the only way to represent it in mainframe code at the time. Both editions of Mike Cowlishaw's first book on the language use all-caps, *REXX*, although the cover graphic uses mixed case. His book on NetRexx uses mixed case but all caps in the cover graphic with large and small caps, **NETREXX**. An expansion that matches the abbreviation, *REstructured eXtended eXecutor*, was used for the system product in 1984.^[12] The name *Rexx* (mixed case) is used in this article, and is commonly used elsewhere.

Attributes

Objective and subjective attributes of Rexx include:

- Simple syntax
- Ability to route commands to multiple environments
- Ability to support functions, procedures and commands associated with a specific invoking environment.
- Built-in stack with the ability to interoperate with the host stack if there is one
- Small instruction set
- Free-form syntax; indentation is optional but can help readability
- Case-insensitive tokens, including variable names
- Character string basis
- Dynamic data typing; no declarations
- No reserved keywords, except in local context^[b]
- No include file facility
- Arbitrary-precision arithmetic
- Decimal arithmetic, floating-point
- Rich selection of built-in functions, especially string and word processing
- Automatic storage management
- Crash protection
- Content addressable data structures
- Associative array
- Straightforward access to system commands and facilities
- Simple error-handling, and built-in tracing and debugger
- Few artificial limitations
- Simplified I/O facilities
- Unconventional operators
- Only partly supports Unix style command line parameters, except specific implementations
- Provides no basic terminal control as part of the language, except specific implementations
- Provides no generic way to include functions and subroutines from external libraries, except specific implementations

Some claim that Rexx is a relatively simple language. With only 23 instructions (such as `call`, `parse`, and `select`), it has a relatively small instruction set. Rexx has limited punctuation and formatting requirements. Rexx has only one data type, the character string. Some claim that such simplicities make Rexx relatively easy to debug.

Some claim that Rexx code looks similar to PL/I code, but has fewer notations. With fewer notations, it tends to be is harder to parse via a translator, but is easier to write. Simplifying coding was intentional as noted by the Rexx design goal of the principle of least astonishment.^[12]

History

pre-1990

On his own time, Mike Cowlshaw developed the language and an interpreter for it in assembly language between 20 March 1979 and mid-1982 with the intent to replace the languages EXEC and EXEC 2.^[6] Mike also intended Rexx to be a simplified and easier to learn version of PL/I, but some claim that Rexx has problematic differences from PL/I.

Rexx was first described in public at the SHARE 56 conference in Houston, Texas, in 1981,^[13] where customer reaction, championed by Ted Johnston of SLAC, led to it being shipped as an IBM product in 1982.

Over the years IBM included Rexx in almost all of its operating systems (VM/CMS, MVS TSO/E, IBM OS/400, VSE/ESA, MUSIC/SP, AIX, PC DOS, and OS/2), and has made versions available for Novell NetWare, Windows, Java, and Linux.

The first non-IBM version was written for PC DOS by Charles Daney in 1984/5^[7] and marketed by the Mansfield Software Group (founded by Kevin J. Kearney in 1986).^[6] The first Rexx compiler appeared in 1987, written for CMS by Lundin and Woodruff.^[14] Other versions have also been developed for Atari, AmigaOS, Unix (many variants), Solaris, DEC, Windows, Windows CE, Pocket PC, DOS, Palm OS, QNX, OS/2, Linux, BeOS, EPOC32/Symbian, AtheOS, OpenVMS,^{[15]:309} Apple Macintosh, and Mac OS X.^[16]

ARexx, a Rexx interpreter for Amiga, was included with AmigaOS 2 onwards and was popular for scripting and application control. Many Amiga applications have an "ARexx port" which allows control of the application via a Rexx script. Notably, a Rexx script can switch between Rexx ports to control multiple applications.

1990 to present

In 1990, Cathie Dager of SLAC organized the first independent Rexx symposium, which led to the forming of the Rexx Language Association. Symposia are held annually.

In 1992, the two most widely used open-source ports appeared: Ian Collier's REXX/imc for Unix and Anders Christensen's Regina^[5] (later adopted by Mark Hessling) for Windows and Unix. BRexx was developed by Vasilis N Vlachoudis, a nuclear scientist at CERN. It runs on a range of operating systems,

including Unix, Linux, BSD, macOS and Windows. Its small size means it can run on an Android mobile phone. BRexx/370 is a version that runs on IBM mainframes.^{[15]:359–383}^[17]

OS/2 has a visual development system from Watcom VX-REXX. Another dialect was VisPro REXX from Hockware.

Portable REXX by Kilowatt and *Personal REXX* by Quercus are two REXX interpreters designed for DOS and can be run under Windows as well using a command prompt. Since the mid-1990s, two newer variants of REXX have appeared:

- **NetREXX**: compiles to Java byte-code via Java source code; this has no reserved keywords at all, and uses the Java object model, and is therefore not generally upwards-compatible with 'classic' REXX.
- **Object REXX**: an object-oriented generally upwards-compatible version of REXX.

In 1996 the American National Standards Institute (ANSI) published a standard for REXX: ANSI X3.274–1996 "Information Technology – Programming Language REXX".^[c] More than two dozen books on REXX have been published since 1985.

REXX marked its 25th anniversary on 20 March 2004, which was celebrated at the REXX Language Association's 15th International REXX Symposium in Böblingen, Germany, in May 2004.

On October 12, 2004, IBM announced their plan to release their Object REXX implementation's sources under the Common Public License. Recent releases of Object REXX contain an ActiveX Windows Scripting Host (WSH) scripting engine implementing this version of the REXX language.

On February 22, 2005, the first public release of Open Object REXX (ooREXX) was announced. This product contains a WSH scripting engine which allows for programming of the Windows operating system and applications with REXX in the same fashion in which Visual Basic and JScript are implemented by the default WSH installation and Perl, Tcl, Python third-party scripting engines.

In January 2018 the TIOBE index listed REXX at position 30.^[19] Since 2018 it has been either outside the top 50, or, more frequently, outside the top 100.

In 2019, the 30th REXX Language Association Symposium marked the 40th anniversary of REXX. The symposium was held in Hursley, England, where REXX was first designed and implemented.^[20]

Toolkits

- **REXXUtil** – a package of file and directory functions, windowed I/O, and functions to access system services such as WAIT and POST – is available for most REXX environments.^[21]^[22]^[23]
- **REXX/Tk** – a toolkit for graphics to be used in REXX programmes in the same fashion as Tcl/Tk – is widely available.
- **REXXEd** – an integrated development environment (IDE) for REXX – was developed for Windows.^{[15]:390}
- **RxSock** for network communication as well as other add-ons to and implementations of Regina REXX have been developed, and a REXX interpreter for the Windows command line is supplied in most resource kits for various versions of Windows and works in DOS as well.

Host environment

A Rexx host environment is a named interface for sending commands to an, e.g., application, operating system, subsystem. The Rexx interpreter initially has a list of known environments; the first in the list is known as the *default environment*. A Rexx script use the ADDRESS statement to change the default environment and may also use it to send a single command to a specific environment without changing the default environment.

Syntax

Instruction types

Rexx has three instruction types^{[24]:31}

- Assignments – Single clause of the form *symbol=expression*, assigns a value to a variable; e.g. `count=count+1`, would add 1 to value in variable "count".
- Keyword instructions – The keyword is the first word of the instruction; e.g. **say** "Message", would print the word 'Message'.
- Commands – an expression that the interpreter evaluates and passes as a command to the default environment; e.g. `"sleep 10"` would cause a Unix-like host environment to produce a ten seconds delay.

Address instruction

The address instruction has three roles:

- An address instruction with no operand interchanges the first two host environments; normally this reinstate the previous default environment.
- An address instruction with only an environment moves it to the beginning of the list, making it the default environment.
- An address instruction with an environment and an expression passes the value of the expression as a command to the specified environment.

The ANSI standard added options for redirecting the input and output of commands.

Do groups

The language provides do groups for two purposes:

- To treat a group of instructions within an `if` or `select` statement as a unit for purposes of flow control.
- For loop control, similar to many other languages. A single do group may optionally contain repetitor phrases, conditional phrases, or both, with termination whenever any of them is satisfied.

A **do** group begins with **do** and ends with **end**. A single group may serve both purposes. In the related ooRexx and NetRexx, there is both a **do** and a **loop** keyword, with almost identical semantics; they differ in that a simple **do** is equivalent to **do 1** while a simple **loop** is equivalent to **loop forever**.

An iteration of a **do** group may be terminated with an **iterate** statement and the entire group may be terminated with a **leave** statement.

Simple do

Although it is valid anywhere, a simple do is specifically useful inside conditional statements:

```
if foo=bar
then
  do
    i=1;j=3
  end
else
  do
    i=2;j=4
  end
```

Conditional loop

The language supports testing a condition either before (**do while**) or after (**do until**) executing a group of code via syntax:

```
do while [condition]
  [instructions]
end
```

```
do until [condition]
  [instructions]
end
```

Simple repetitive loop

The language permits counted loops, where an expression is computed at the start of the loop and the instructions within the loop are executed that many times:

```
do expression
  [instructions]
end
```

Controlled Repetitive Loops

A loop can increment a variable and stop when a limit is reached.

```
do index=start [to limit] [by increment] [for count]
  [instructions]
end
```

The increment value is 1 if the **by** clause is omitted. The loop continues forever if the limit **to** clause is omitted, unless terminated earlier by another clause or by a **leave** statement.

Unconditional loop

The language supports an unconditional loop via `forever` that continues until the loop is terminated or the program is terminated.

```
do forever
  [instructions]
end
```

Combined loop

Repetitive elements and conditional phrases can be combined in the same loop;^{[24]:50} e.g.:

```
do i=1 while i<=3; say i; end
```

```
do index=start [to limit] [by increment] [for count] [while condition]
  [instructions]
end
```

```
do expression [until condition]
  [instructions]
end
```

Conditional

The language provides for conditional execution via `if`, `then` and `else` for a group delimited by `do` and `end`.

```
if [condition] then
  do
    [instructions]
  end
else
  do
    [instructions]
  end
```

For a single instruction, `do` and `end` can be omitted.

```
if [condition] then
  [instruction]
else
  [instruction]
```

Multiple condition branching

The language provides multiple condition branching via `select` which derives from the `SELECT`; form of the PL/I `SELECT` statement ^[d] Like similar constructs in other dynamic languages, Rexx's `when` clauses specify full conditions – not equality tests of a single value for the statement as some languages do. In that, they are more like cascading `if-then-else` code than like the C or Java `switch` statement.

```

select
  when [condition] then
    [instruction or nop]
  when [condition] then
    do
      [instructions or nop]
    end
  otherwise
    [instructions or nop]
end

```

The `nop` instruction is required if no action is associated with a `when` condition.

The `otherwise` clause is optional. If omitted and no `when` conditions are met, then the syntax condition is raised.

Variable

Typing system

Variables are typeless and initially are evaluated as their names in upper case. Thus a variable's type can vary with its use in the program:

```

say hello /* => HELLO */
hello = 25
say hello /* => 25 */
hello = "say 5 + 3"
say hello /* => say 5 + 3 */
interpret hello /* => 8 */
drop hello
say hello /* => HELLO */

```

Evaluation

If no `novalue` condition handler is configured, then an undefined variable evaluates to its name, in upper case. The built-in function `SYMBOL` returns "VAR" for a defined variable and does *not* trigger `novalue` even if not defined. The `VALUE` function gets the value of a variable without triggering a `novalue` condition, but its main purpose is to read and set environment variables, similar to POSIX `getenv` and `putenv`.

Compound variable

The language provides the **compound variable** construct which supports adding fields (called tails) to a variable (called a stem in this context) to support data structures such as lists, arrays, n-dimensional arrays, sparse or dense arrays, balanced or unbalanced trees and records.

The language does not provide special support for numeric array indexing like many other languages do. Instead, a compound variable with numeric tails produce a similar effect.^[25]

The following code defines variables `stem.1 = 9`, `stem.2 = 8`, `stem.3 = 7`...

```

do i = 1 to 10
  stem.i = 10 - i
end

```



```
end
```

Unlike a typical array, a tail (index) need not identify (be named) an integer value. For example, the following code is valid:

```
i = 'Monday'
stem.i = 2
```

A default value can be assigned to a stem via `.` but no tail.

```
stem. = 'Unknown'
stem.1 = 'USA'
stem.44 = 'UK'
stem.33 = 'France'
```

In this case `stem.3`, for example, evaluates to the default value, `'Unknown'`.

The whole stem (including any default value) can be erased with the `drop` statement.

```
drop stem.
```

By convention (not part of the language) the compound `stem.0` is often used to keep track of how many items are in a stem, for example a procedure to add a word to a list might be coded like this:

```
add_word: procedure expose dictionary.
  parse arg w
  n = dictionary.0 + 1
  dictionary.n = w
  dictionary.0 = n
return
```

A stem can have multiple tails. For example:

```
m = 'July'
d = 15
y = 2005
day.y.m.d = 'Friday'
```

Multiple numerical tail elements can be used to provide the effect of a multi-dimensional array.

Features similar to the compound variable are found in other languages including associative arrays in AWK, hashes in Perl and hash tables in Java, dynamic objects in JavaScript. Most of these languages provide a mechanism to iterate over the keys (tails) of such a construct, but this is lacking in classic Rexx. Instead, it is necessary to store additional information. For example, the following procedure might be used to count each occurrence of a word.

```
add_word: procedure expose count. word_list
  parse arg w .
  count.w = count.w + 1 /* assume count. has been set to 0 */
  if count.w = 1 then word_list = word_list w
return
```

and then later:

```
do i = 1 to words(word_list)
  w = word(word_list,i)
  say w count.w
end
```

More recent Rexx variants, including Object REXX and ooRexx, provide a construct to iterate over the tails of a stem.

```
do i over stem.
  say i '-->' stem.i
end
```

Parse

The parse instruction provides string-handling via syntax:

parse [**upper**] *origin* [*template*]

If **upper** is included then the input is converted to upper case before parsing.

origin describes the input as one of the following:

- **arg** – arguments, at top level tail of command line
- **linein** – standard input, e.g. keyboard
- **pull** – Rexx data queue or standard input
- **source** – info on how program was executed
- value *expression* with – with indicates the end of the expression
- **var** – a variable
- **version** – version/release number

template can be a combination of variables, literal delimiters, and column number delimiters.

Examples

Using a list of variables as template:

```
myVar = "John Smith"
parse var myVar firstName lastName
say "First name is:" firstName
say "Last name is:" lastName
```

displays:

```
First name is: John
Last name is: Smith
```

Using column number delimiters:

```
myVar = "(202) 123-1234"
parse var MyVar 2 AreaCode 5 7 SubNumber
say "Area code is:" AreaCode
say "Subscriber number is:" SubNumber
```

displays:

```
Area code is: 202
Subscriber number is: 123-1234
```

Interpret

The `interpret` instruction evaluates its argument as a Rexx statement allowing for evaluation of code formatted at runtime. Uses include passing a function as a parameter, arbitrary precision arithmetic, use of the `parse` statement with programmatic templates, stemmed arrays, and sparse arrays. The following example displays 16 and exits.

```
X = 'square'
interpret 'say' X || '(4) ; exit'
SQUARE: return arg(1)**2
```

The Valour software package relied upon Rexx's interpretive ability to implement an OOP environment. Another use was found in an unreleased Westinghouse product called *Time Machine* that was able to fully recover following an otherwise fatal error.

Numeric

```
say digits() fuzz() form() /* => 9 0 SCIENTIFIC */
say 999999999+1 /* => 1.000000000E+9 */
numeric digits 10 /* only limited by available memory */
say 999999999+1 /* => 1000000000 */

say 0.999999999=1 /* => 0 (false) */
numeric fuzz 3
say 0.999999999=1 /* => 1 (true) */
say 0.999999999==1 /* => 0 (false) */

say 100*123456789 /* => 1.23456789E+10 */
numeric form engineering
say 100*123456789 /* => 12.34567890E+9 */

say 53 // 7 /* => 4 (rest of division)*/
```

Calculation of the value $\sqrt{2}$:

```
numeric digits 50
n=2; r=1
do forever /* Newton's method */
  rr=(n/r+r)/2
  if r=rr then leave
  r=rr
end
say "SqRt" n "=" r
```

Output: SqRt 2 = 1.414213562373095048801688724209698078569671875377

Calculation of the value e :

```
numeric digits 50
e=2.5; f=0.5
do n=3
  f=f/n
  ee=e+f
  if e=ee then leave
  e=ee
end
say "e =" e
```

Output: e = 2.7182818284590452353602874713526624977572470936998

Error handling

The `signal` instruction configures the runtime to run custom code to handle a system condition if triggered. Conditions include:

- `error` – Positive return code from a system command
- `failure` – Negative return code from a system command (e.g. command doesn't exist)
- `halt` – Abnormal termination
- `novalue` – A variable name was used but the variable is not defined
- `notready` – Input or output error (e.g. read attempts beyond end of file)
- `syntax` – Invalid program syntax, or some other error condition
- `lostdigits` – Significant digits were lost (ANSI Rexx, not in TRL second edition)

The following fragment prints a message when the user terminates (halts) it:

```
signal on halt;
do a = 1
  say a
  do 100000 /* a delay */
  end
end
halt:
say "The program was stopped by the user"
exit
```

Since Rexx version 4, a handler can be named. In the following example, the handler `ChangeCodePage.Trap` is configured to handle a `syntax` condition.

```
ChangeCodePage: procedure
  signal on syntax name ChangeCodePage.Trap
  return SysQueryProcessCodePage()

ChangeCodePage.Trap: return 1004
```

When a condition is handled (as configured via `signal on`), the condition can be analyzed via `RC` which indicates the last error code and `SIGL` which indicates the line number of the code that triggered the condition.

See also



**Free and open-
source software
portal**

- [ISPF](#)
- [XEDIT](#)
- [Comparison of computer shells](#)
- [Comparison of programming languages](#)

Notes

- The TSO EXEC command with an unqualified dataset name, and neither the CLIST nor EXEC option, examines the low level qualifier for "EXEC".
- It is best practice to avoid using keywords as labels or names.
- While ANSI INCITS 274-1996/AMD1-2000 (R2001) and ANSI INCITS 274-1996 (R2007) are chargeable, a free draft can be downloaded.^[18]
- Rexx has no equivalent to the `SELECT (expression);` form.

References

- [Virtual Machine/System Product - System Product - Interpreter Reference - Release 3](http://bitsavers.org/pdf/ibm/370/VM/SP/Release_3.0_Jul83/SC24-5239-0_VM_SP_Interpreter_Reference_Rel_3_Sep83.pdf) (http://bitsavers.org/pdf/ibm/370/VM/SP/Release_3.0_Jul83/SC24-5239-0_VM_SP_Interpreter_Reference_Rel_3_Sep83.pdf) (PDF) (First ed.). IBM. September 1983. SC24-5239-0. Retrieved November 6, 2024.
- [TSO Extensions Version 2 - Procedures Language MVS/REXX Reference](http://bitsavers.org/pdf/ibm/370/TSO_Extensions/SC28-1883-4_TSO_Extensions_Version_2_Procedures_Language_MVS_REXX_Reference_Aug1991.pdf) (http://bitsavers.org/pdf/ibm/370/TSO_Extensions/SC28-1883-4_TSO_Extensions_Version_2_Procedures_Language_MVS_REXX_Reference_Aug1991.pdf) (PDF) (Fifth ed.). IBM. August 1991. SC28-1883-4. Retrieved November 6, 2024.
- ["Procedures Language"](http://bitsavers.org/pdf/ibm/saa/GC26-4341-0_System_Application_Architecture_An_Overview_198705.pdf) (http://bitsavers.org/pdf/ibm/saa/GC26-4341-0_System_Application_Architecture_An_Overview_198705.pdf) (http://bitsavers.org/pdf/ibm/saa/GC26-4341-0_System_Application_Architecture_An_Overview_198705.pdf) (PDF) (First ed.). IBM. May 1987. p. 40. GC26-4341-0. Retrieved November 6, 2024.
- [Procedures Language/2 - REXX Reference - Version 2.00](http://bitsavers.org/pdf/ibm/pc/os2/OS2_2.x/IBM_OS2_2.0_Technical_Library_1992/S10G-6268-00_OS2_2.0_Procedures_Language_2_REXX_Reference_199203.pdf) (http://bitsavers.org/pdf/ibm/pc/os2/OS2_2.x/IBM_OS2_2.0_Technical_Library_1992/S10G-6268-00_OS2_2.0_Procedures_Language_2_REXX_Reference_199203.pdf) (PDF). OS/2 2.00 - Technical Library (First ed.). IBM. December 1991. S10G-6268-00. Retrieved November 6, 2024.
- Mark Hessling (October 25, 2012). "Regina REXX Interpreter" (<http://regina-rexx.sourceforge.net>). SourceForge project regina-rexx. Retrieved February 10, 2014.
- M. F. Cowlshaw. "IBM REXX Brief History" (http://www.rexxla.org/links/IBM_historical_pages/rexxhist.html). IBM. Retrieved August 15, 2006.
- Melinda Varian. "REXX Symposium, May 1995" (<http://www.rexxla.org/events/1995/report.html>).

8. "Catalog of All Documents (filter=rexx)" (https://web.archive.org/web/20130215214058/http://publibz.boulder.ibm.com/cgi-bin/bookmgr_OS390/FINDBOOK?filter=rexx). IBM library server. 2005. Archived from the original (http://publibz.boulder.ibm.com/cgi-bin/bookmgr_OS390/FINDBOOK?filter=rexx) on February 15, 2013. Retrieved February 10, 2014.
9. "Does ArcaOS include REXX support?" (<https://www.arcanos.com/faqwd/does-arcaos-include-rexx-support>). Retrieved September 3, 2020.
10. *IBM Virtual Machine Facility /370: EXEC User's Guide* (http://bitsavers.org/pdf/ibm/370/VM/370/Release_2/GC20-1812-1_VM370_EXEC_Users_Guide_Rel_2_Apr75.pdf) (PDF) (Second ed.). International Business Machines Corporation. April 1975. GC20-1812-1.
11. *EXEC 2 Reference* (https://web.archive.org/web/20200402040956/http://bitsavers.org/pdf/ibm/370/VM_SP/Release_2_Jun82/SC24-5219-1_VM_SP_EXEC_2_Rel_2_Reference_Apr82.pdf) (PDF) (Second ed.). International Business Machines Corporation. April 1982. p. 92. SC24-5219-1. Archived from the original (http://bitsavers.org/pdf/ibm/370/VM_SP/Release_2_Jun82/SC24-5219-1_VM_SP_EXEC_2_Rel_2_Reference_Apr82.pdf) (PDF) on April 2, 2020. Retrieved March 28, 2019.
12. M. F. Cowlshaw (1984). "The design of the REXX language" (<https://www.cs.tufts.edu/~nr/cs257/archive/mike-cowlshaw/rexx.pdf>) (PDF). *IBM Systems Journal* (PDF). **23** (4). IBM Research: 333. doi:10.1147/sj.234.0326 (<https://doi.org/10.1147/sj.234.0326>). Retrieved January 23, 2014. "Could there be a high astonishment factor associated with the new feature? If a feature is accidentally misapplied by the user and causes what appears to him to be an unpredictable result, that feature has a high astonishment factor and is therefore undesirable. If a necessary feature has a high astonishment factor, it may be necessary to redesign the feature."
13. M. F. Cowlshaw (February 18, 1981). "REX -- A Command Programming Language" (<http://speleotrove.com/rexxhist/share56.txt>). SHARE. Retrieved August 15, 2006.
14. Lundin, Leigh; Woodruff, Mark (April 23, 1987). "T/REXX, a REXX compiler for CMS" (<http://web.archive.org/web/20160303204104/http://www.copyrightsearch.org/trexx-a-rexx-compiler-for-cms/TXu000295377>). *U.S. Copyright Office* (TXu000295377). Washington, DC: Independent Intelligence Incorporated. Archived from the original (<http://www.copyrightsearch.org/trexx-a-rexx-compiler-for-cms/TXu000295377>) on March 3, 2016. Retrieved February 20, 2010.
15. Fosdick, Howard (2025). *Rexx Programmer's Reference* (https://rexxinfo.org/Rexx_Programmers_Reference.pdf) (PDF) (2 ed.). Apex, North Carolina: Rexx Language Association. ISBN 9789403745527. OCLC 1475017186 (<https://search.worldcat.org/oclc/1475017186>). Archived (https://web.archive.org/web/20250403173024/https://rexxinfo.org/Rexx_Programmers_Reference.pdf) (PDF) from the original on April 3, 2025. Retrieved May 26, 2025.
16. "Rexx Implementations" (https://web.archive.org/web/20060924072512/http://www.rexxla.org/About_Rexx/mfc/rexxplat.html). RexxLA. Archived from the original (http://www.rexxla.org/About_Rexx/mfc/rexxplat.html) on September 24, 2006. Retrieved August 15, 2006.
17. Vlachoudis, Vassilis (June 2011). Written at CERN. "BRexx" (<https://github.com/vlachoudis/brexx>). San Francisco, California: GitHub. Archived (<https://web.archive.org/web/20250612085939/https://github.com/vlachoudis/brexx>) from the original on June 12, 2025. Retrieved June 24, 2025.
18. "American National Standard for Information Systems – Programming Language REXX" (<https://www.rexxla.org/rexxlang/standards/j18pub.pdf>) (PDF). *The Rexx Language Association*. Washington D.C.: American National Standards Institute (ANSI). X3J18-199X. Archived (<https://web.archive.org/web/20250403153402/https://www.rexxla.org/rexxlang/standards/j18pub.pdf>) (PDF) from the original on April 3, 2025. Retrieved June 7, 2025.
19. "TIOBE Index for January 2018" (<https://web.archive.org/web/20180203212305/https://www.tiobe.com/tiobe-index#:~:text=0.569%,-30,0.562%,-31>). *TIOBE*. Eindhoven, Netherlands. January 2018. Retrieved June 7, 2025.
20. "RexxLA - Symposium Schedule" (<https://www.rexxla.org/events/schedule.rsp?year=2019>).

21. Ashley, W David; Flatscher, Rony G; Hessling, Mark; McGuire, Rick; Peedin, Lee; Sims, Oliver; Steinbock, Erich; Wolfers, Jon (January 19, 2024). "8. REXX Utilities (RexxUtil)" (<https://rexxinfo.org/reference/articles/oorexxref.pdf#page=530>) (PDF). *Open Object REXX: Reference* (<https://rexxinfo.org/reference/articles/oorexxref.pdf>) (PDF) (5.1.0 ed.). REXX Language Association. pp. 512–566. Archived (<https://web.archive.org/web/20240525080605/https://rexxinfo.org/reference/articles/oorexxref.pdf#page=530>) (PDF) from the original on May 25, 2024. Retrieved May 26, 2025.
22. "REXX Tips & Tricks:REXXUTIL functions" (http://www.edm2.com/index.php/REXX_Tips_%26_Tricks:REXXUTIL_functions). *EDM2: The Electronic Developer Magazine for OS/2*. Retrieved October 14, 2023.
23. "Regina REXX Interpreter" (<https://regina-rexx.sourceforge.io/>). *Sourceforge*. Retrieved October 14, 2023.
24. Cowlshaw, Michael (1990). *The REXX Language: A Practical Approach to Programming* (https://rexxinfo.org/info/articles/the_rexx_language_2nd_edition.pdf) (PDF) (2 ed.). Englewood Cliffs, New Jersey: Prentice Hall. ISBN 9780137806515. LCCN 89071130 (<https://lccn.loc.gov/89071130>). OCLC 20826616 (<https://search.worldcat.org/oclc/20826616>). OL 2226199M (<https://openlibrary.org/books/OL2226199M>). Archived (https://web.archive.org/web/20240421142252/https://rexxinfo.org/info/articles/the_rexx_language_2nd_edition.pdf) (PDF) from the original on April 21, 2024. Retrieved June 13, 2025.
25. "How to Code Arrays and Other Data Structures In REXX" (https://rexxinfo.org/info/articles/how_to_code_arrays.pdf) (PDF).

Further reading

- Callaway, Merrill. *The ARexx Cookbook: A Tutorial Guide to the ARexx Language on the Commodore Amiga Personal Computer*. Whitestone, 1992. ISBN 978-0963277305.
- Callaway, Merrill. *The REXX Cookbook: A Tutorial Guide to the REXX Language in OS/2 & Warp on the IBM Personal Computer*. Whitestone, 1995. ISBN 0-9632773-4-0.
- Cowlshaw, Michael. *The REXX Language: A Practical Approach to Programming*. Prentice Hall, 1990. ISBN 0-13-780651-5.
- Cowlshaw, Michael. *The NetREXX Language*. Prentice Hall, 1997. ISBN 0-13-806332-X.
- Daney, Charles. *Programming in REXX*. McGraw-Hill, TX, 1990. ISBN 0-07-015305-1.
- Ender, Tom. *Object-Oriented Programming With REXX*. John Wiley & Sons, 1997. ISBN 0-471-11844-3.
- Fosdick, Howard (2025). *REXX Programmer's Reference* (https://rexxinfo.org/REXX_Programmers_Reference.pdf) (PDF) (2 ed.). Apex, North Carolina: REXX Language Association. ISBN 9789403745527. OCLC 1475017186 (<https://search.worldcat.org/oclc/1475017186>). Archived (https://web.archive.org/web/20250403173024/https://rexxinfo.org/REXX_Programmers_Reference.pdf) (PDF) from the original on April 3, 2025. Retrieved May 3, 2025.
- Gargiulo, Gabriel. *REXX with OS/2, TSO, & CMS Features*. MVS Training, 1999 (third edition 2004). ISBN 1-892559-03-X.
- Goldberg, Gabriel and Smith, Philip H. *The REXX Handbook*. McGraw-Hill, TX, 1992. ISBN 0-07-023682-8.
- Goran, Richard K. *REXX Reference Summary Handbook*. CFS Nevada, Inc., 1997. ISBN 0-9639854-3-4.
- IBM Redbooks. *Implementing REXX Support in Sdsf*. Vervante, 2007. ISBN 0-7384-8914-X.
- Kiesel, Peter C. *REXX: Advanced Techniques for Programmers*. McGraw-Hill, TX, 1992. ISBN 0-07-034600-3.
- Marco, Lou *ISPF/REXX Development for Experienced Programmers*. CBM Books, 1995. ISBN 1-878956-50-7

- O'Hara, Robert P. and Gomberg, David Roos. *Modern Programming Using Rexx*. Prentice Hall, 1988. [ISBN 0-13-597329-5](#).
- Rudd, Anthony S. 'Practical Usage of TSO REXX'. CreateSpace, 2012. [ISBN 978-1475097559](#).
- Schindler, William. *Down to Earth Rexx*. Perfect Niche Software, 2000. [ISBN 0-9677590-0-5](#).

External links

- Mike Cowlshaw's home page (<https://speleotrove.com/mfc/mfc.html>)
 - Rexx Language Association (<https://www.rexxla.org/>)
 - Rexxinfo.org (<https://rexinfo.org/>)
 - The Rexx Language (Background) (<https://speleotrove.com/rexxhist/TRL-background.pdf>)
 - REXX programming language (<https://www.ibm.com/docs/en/zos/2.5.0?topic=programming-rexx-language>) language reference at IBM
 - Rexx Programmer's Reference 2nd Edition (https://rexinfo.org/Rexx_Programmers_Reference.pdf)
-

Retrieved from "<https://en.wikipedia.org/w/index.php?title=Rexx&oldid=1298548763>"